

Modernizing Enterprise CI/CD

Architectural Proposal: Shifting from Legacy GitFlow to Artifact Promotion

1. Executive Summary

As we standardise the CI pipeline for our Maven-based applications across the business line, we have an opportunity to align our delivery model with modern Cloud-Native and DevOps best practices. This document proposes moving away from the "Textbook GitFlow" Maven versioning model in favor of the industry-standard **Artifact Promotion Model** (Build Once, Deploy Anywhere).

While strict GitFlow branching is valuable for source control management, tightly coupling Maven version lifecycles (e.g., automated POM modifications and recompilations on the main branch) introduces significant rebuild risks and breaks artifact immutability within an automated CI/CD ecosystem.

2. The Problem: Textbook GitFlow vs. Maven Immutability

In the traditional textbook approach, the CI pipeline modifies the `pom.xml` based on the branch and triggers a Maven build (`mvn package`). The flow typically looks like this:

- **Develop:** `1.1.0-SNAPSHOT` (Build & Deploy to Dev)
- **Release Branch:** Pipeline changes POM to `1.1.0-RC1` (Build & Deploy to QA)
- **Main Branch:** After QA sign-off, merge to Main. Pipeline changes POM to `1.1.0` (Build & Deploy to Prod)

The Critical Flaw: By changing the POM version to `1.1.0` upon merging to main, the CI pipeline is forced to execute a new `mvn compile` and `mvn package`.

This violates the DevOps principle of *Artifact Immutability*. The binary deployed to Production is **not** the exact binary that QA signed off on. Even if the source code is identical, a new build introduces the risk of transient dependency drift, different timestamps, and build-environment anomalies. You are deploying an untested artifact.

3. The Cons of Legacy GitFlow in CI/CD

Adhering strictly to Maven version bumping and recompiling on the main branch introduces critical operational risks:

- **Risk of Regression:** Rebuilding artifacts on the main branch invalidates previous QA integration and security testing.
- **Build Environment Drift:** Recompiling on the main branch means relying on the CI environment's state at two different points in time. For example, a release branch binary might be built on a CI node with OS Security Patch 1.2, while the subsequent main branch build happens on a node still running Patch 1.0. If the application has underlying OS dependencies, this silent drift can introduce an untested regression directly into Production.

4. The Proposed Solution: Artifact Promotion Model

Modern enterprise DevOps teams decouple the *build* process from the *release orchestration* process. The CI pipeline generates immutable binaries on the release branch, and the orchestration pipeline promotes them using JFrog Artifactory.

How It Works:

1. **Branch Creation:** The `develop` branch remains at `1.1.0-SNAPSHOT`. When a release branch is automatically created, the pipeline updates the `pom.xml` strictly to the final release version: `1.1.0`.
2. **Iterative Build (On Release Branch):** The CI pipeline builds the `1.1.0` package and uploads it to a staging/development JFrog repository (e.g., `libs-local-dev`).
3. **QA Validation & Overrides:** QA tests the `1.1.0` package. If bugs are found, developers commit fixes to the release branch. The CI pipeline runs again, regenerating the `1.1.0` package and *overwriting* the previous artifact in the `libs-local-dev` JFrog repository.
4. **Promotion (From Release Branch):** Once QA completes validation, the Release Manager triggers the release pipeline directly from the release branch. **This pipeline does not compile code.** It executes an API call to JFrog to promote (move/copy) the latest, tested `1.1.0` package from the `-local-dev` repository to the final Production release repository.
5. **Merge to Main:** The code is merged to `main` purely for Git history and tagging. No CI compilation runs on the `main` branch.

5. Comparison Matrix

Criteria	Textbook GitFlow	Proposed Artifact Promotion (JFrog)
Artifact Immutability	Failed. Recompiles on Main branch.	Guaranteed. QA-tested binary is promoted directly to PRD.
Production Risk	High. Susceptible to build-environment drift and dependency shifts.	Zero. The exact bytecode tested in QA goes to Production.
Release Pipeline Execution	Heavy. Requires full Maven lifecycle download, compile, test, and package.	Lightweight. Executes a fast, reliable JFrog API promotion call.
Main Branch Role	Active build environment.	Purely historical record and tagging trigger.

6. Conclusion

Adopting this Artifact Promotion model resolves the fundamental clash between Maven versioning and DevOps immutability. By isolating iterative artifact generation to a JFrog staging repository and restricting the final release pipeline to a pure promotion action, we provide Enterprise Architecture with the mathematical guarantee that Production deployments are rigorously tested, traceably secure, and free from regression risks.